

UNIVERSIDAD NACIONAL
AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA



LABORATORIO 09

MEXI BANCO

MENSAJERÍA CON RABBITMQ

Que el aviso no tumbe la transferencia

Bitácora · Sistemas Distribuidos

DATOS ACADÉMICOS

Alumna	Estrella de Maria Mendoza Camacho
No. de cuenta	321132927
Profesor	Oscar Manuel Ruiz Hurtado
Grupo	2
Semestre	2027-1

1 Problema inicial

El objetivo de esta práctica fue modificar Mexi Banco para que las transferencias pudieran terminar correctamente aunque el servicio Notificación estuviera apagado.

Al inicio, Transferencia enviaba los avisos mediante una llamada HTTP directa. Esto provocaba que, si Notificación fallaba, la transferencia también mostrara un error, aunque el dinero ya se hubiera movido.

Para comenzar, se levantaron los servicios con Docker Compose y se revisó el estado inicial de RabbitMQ.

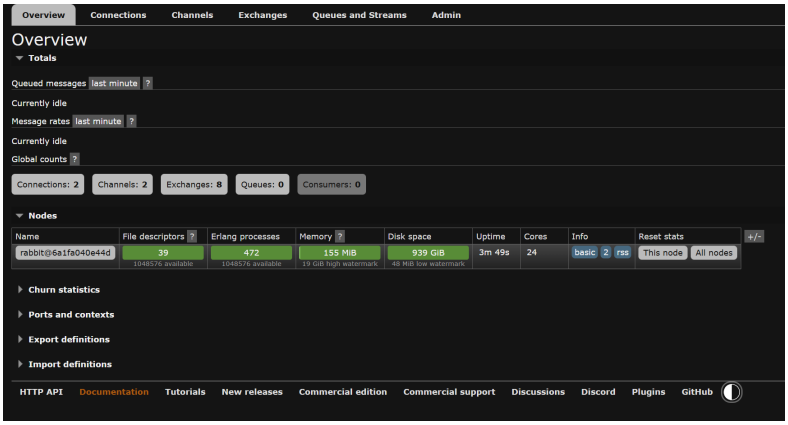


Figura 1: Estado inicial de RabbitMQ antes de configurar la mensajería de transferencias.

Después se crearon dos cuentas para realizar las pruebas: Ana con un saldo inicial de $1,000 y Beto con $500.

```
PS C:\Users\Billie Jean\mexi-banco-s09> $ana = @{
>> clabe = "002180000000000001"
>> titular = "Ana"
>> saldoInicial = 1000
>> } | ConvertTo-Json
PS C:\Users\Billie Jean\mexi-banco-s09> $beto = @{
>> clabe = "002180000000000002"
>> titular = "Beto"
>> saldoInicial = 500
>> } | ConvertTo-Json
PS C:\Users\Billie Jean\mexi-banco-s09> Invoke-RestMethod -Method Post `
>> -Uri "http://localhost:8080/api/cuentas" `
>> -ContentType "application/json" `
>> -Body $ana

id clabe              titular saldo
-- -----              ------- -----
10 002180000000000001 Ana      1000


PS C:\Users\Billie Jean\mexi-banco-s09> Invoke-RestMethod -Method Post `
>> -Uri "http://localhost:8080/api/cuentas" `
>> -ContentType "application/json" `
>> -Body $beto

id clabe              titular saldo
-- -----              ------- -----
11 002180000000000002 Beto      500
```

Figura 2: Creación de las cuentas de Ana y Beto.

1.1 Prueba con Notificación apagado

Se apagó el servicio Notificación y se intentó realizar una transferencia de $50 desde la cuenta de Ana hacia la de Beto.

La solicitud devolvió un error HTTP 500.

```
PS C:\Users\Billie Jean\mexi-banco-s09> docker compose stop notificacion
[+] stop 1/1
 ⠿ Container mexi-banco-v06-notificacion-1 Stopped
PS C:\Users\Billie Jean\mexi-banco-s09> $transferencia = @{
>> claveOrigen = "00218000000000001"
>> claveDestino = "00218000000000002"
>> monto = 50
>> } | ConvertTo-Json
PS C:\Users\Billie Jean\mexi-banco-s09> try {
>> Invoke-WebRequest -Method Post `
>> -Uri "http://localhost:8080/api/transferencias" `
>> -ContentType "application/json" `
>> -Body $transferencia
>> }
>> catch {
>> Write-Host "HTTP:" ([int]$_.Exception.Response.StatusCode)
>> Write-Host $_.ErrorDetails.Message
>> }
HTTP: 500
```

Figura 3: Error HTTP 500 al realizar una transferencia con Notificación apagado.

Sin embargo, al consultar las cuentas, Ana tenía \$950 y Beto \$550. Esto mostró que el dinero sí se había movido.

```
PS C:\Users\Billie Jean\mexi-banco-s09> Invoke-RestMethod `
>> -Uri "http://localhost:8080/api/cuentas/002180000000000001"

id clabe              titular  saldo
-- -----              -------  -----
10 002180000000000001 Ana     950.00


PS C:\Users\Billie Jean\mexi-banco-s09> Invoke-RestMethod `
>> -Uri "http://localhost:8080/api/cuentas/002180000000000002"

id clabe              titular  saldo
-- -----              -------  -----
11 002180000000000002 Beto    550.00
```

Figura 4: Los saldos cambiaron a pesar del error en la transferencia.

También se revisó el estado de la transferencia y se encontró como `ABONADA`, en lugar de `COMPLETADA`.

```
PS C:\Users\Billie Jean\mexi-banco-s09> Invoke-RestMethod `
>> -Uri "http://localhost:8080/api/transferencias?estado=ABONADA" |
>> ConvertTo-Json -Depth 6
{
    "value":  [
                  {
                      "id":  5,
                      "claveOrigen":  "002180000000000001",
                      "claveDestino":  "002180000000000002",
                      "monto":  50.00,
                      "fecha":  "2026-10-24T14:08:30.257880Z",
                      "estado":  "ABONADA",
                      "motivo":  null,
                      "saldoOrigenTrasCargo":  950.00,
                      "intentosDeCompensacion":  0,
                      "actualizada":  "2026-10-24T14:08:30.546237Z"
                  }
              ],
    "Count":  1
}
```

Figura 5: Transferencia en estado ABONADA después del error.

Resultado de la prueba

El problema era que Transferencia dependía de Notificación para terminar correctamente. Por eso se decidió utilizar RabbitMQ para enviar los avisos.

2 Publicación de eventos con RabbitMQ

Se modificó el servicio Transferencia para quitar la llamada HTTP directa a Notificación.

Ahora, cuando termina una transferencia, el servicio publica un evento en RabbitMQ. De esta forma, Notificación puede recibir el aviso después, sin detener la operación bancaria.

Se creó el exchange `transferencias`, de tipo `topic` y durable.

```
@Bean
public TopicExchange intercambioTransferencias() {
    return new TopicExchange(
        "transferencias", true, false
    );
}
```

Se utilizaron dos eventos:

- `TransferenciaRealizada`: se publica cuando la transferencia termina correctamente.
- `TransferenciaRevertida`: se publica cuando falla una transferencia y se devuelve el dinero.

Cada evento contiene un `eventoId`, que permite reconocerlo si RabbitMQ vuelve a entregar el mismo mensaje.

También se configuraron los *publisher confirms* para comprobar si RabbitMQ aceptó la publicación del evento.

Esta confirmación no asegura que Notificación ya haya procesado el mensaje; solamente confirma su aceptación por RabbitMQ.

2.1 Prueba de publicación

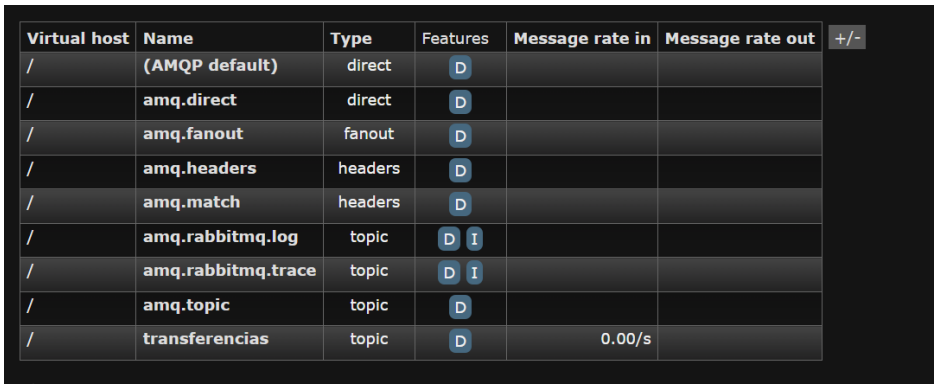
Se volvió a apagar Notificación y se realizó una transferencia por $50.

Esta vez la solicitud respondió HTTP 201 y la transferencia quedó en estado `COMPLETADA`.



Figura 6: Transferencia completada correctamente con Notificación apagado.

También se comprobó que RabbitMQ ya tenía el exchange `transferencias`.



Virtual host	Name	Type	Features	Message rate in	Message rate out	+/-
/	(AMQP default)	direct	D			
/	amq.direct	direct	D			
/	amq.fanout	fanout	D			
/	amq.headers	headers	D			
/	amq.match	headers	D			
/	amq.rabbitmq.log	topic	D I			
/	amq.rabbitmq.trace	topic	D I			
/	amq.topic	topic	D			
/	transferencias	topic	D	0.00/s		

Figura 7: Exchange transferencias creado en RabbitMQ.

Resultado de la publicación

La transferencia pudo terminar sin esperar una respuesta de Notificación. El evento quedó a cargo de RabbitMQ.

3 Recepción de eventos y notificaciones

Se modificó el servicio Notificación para recibir los eventos publicados por Transferencia.

Se creó la cola durable `notificacion.transferencias` y se conectó al exchange mediante el patrón `transferencia.*`.

También se implementó un consumidor con `@RabbitListener`.

Cuando recibe un evento `TransferenciaRealizada`, genera dos avisos: uno para la cuenta de origen y otro para la de destino.

Cuando recibe un evento `TransferenciaRevertida`, genera un aviso de devolución para la cuenta de origen.

Se configuró el ACK manual para confirmar los mensajes después de guardar las notificaciones.

```
List<Notificacion> avisos =
    notificaciones.avisarDeTransferencia(evento);

canal.basicAck(tag, false);
```

3.1 Prueba con tres mensajes pendientes

Se apagó Notificación y se realizaron tres transferencias por $11, $12 y $13.

Todas terminaron correctamente, aunque Notificación no estaba funcionando.

En RabbitMQ aparecieron tres mensajes pendientes, mostrados como `3 Ready`.



Overview					Messages			Message rates	
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get
/	notificacion.transferencias	classic	D Args	running	3	0	3	0.00/s	

Figura 8: Tres mensajes esperando en RabbitMQ mientras Notificación está apagado.

Después se encendió Notificación.

El consumidor recibió los tres mensajes, guardó los avisos y dejó la cola sin mensajes pendientes.

En la bandeja de Beto aparecieron las tres transferencias, con saldos resultantes de $611, $623 y $636.

```
PS C:\Users\Billie Jean\mexi-banco-s09> docker compose exec -T rabbitmq rabbitmqctl list_queues name messages_ready consumers
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  consumers
notificacion.transferencias     0       1
PS C:\Users\Billie Jean\mexi-banco-s09> Invoke-RestMethod `
>> -Uri "http://localhost:8080/api/notificaciones?clabe=00218000000000002" |
>> ConvertTo-Json -Depth 6
{
    "value":  [
                  {
                      "id":  11,
                      "clabeCuenta":  "00218000000000002",
                      "mensaje":  "TRANSFERENCIA_RECIBIDA por 13, saldo resultante 636.00",
                      "fecha":  "2026-10-24T15:27:25.909915Z"
                  },
                  {
                      "id":  9,
                      "clabeCuenta":  "00218000000000002",
                      "mensaje":  "TRANSFERENCIA_RECIBIDA por 12, saldo resultante 623.00",
                      "fecha":  "2026-10-24T15:27:25.895526Z"
                  },
                  {
                      "id":  7,
                      "clabeCuenta":  "00218000000000002",
                      "mensaje":  "TRANSFERENCIA_RECIBIDA por 11, saldo resultante 611.00",
                      "fecha":  "2026-10-24T15:27:25.628321Z"
                  }
              ],
    "Count":  3
}
```

Figura 9: Notificación recibe los tres mensajes guardados en RabbitMQ.

Resultado de la prueba

Los mensajes permanecieron en RabbitMQ mientras Notificación estaba apagado y se procesaron cuando volvió a funcionar.

3.2 Prueba de devolución

También se probó el evento `TransferenciaRevertida`.

Se provocó una falla al intentar abonar $150 a la cuenta de Beto, utilizando la variable `FALLAR_ABONO_A_CLABE`.

Durante la prueba apareció un problema en PostgreSQL: la base de datos no permitía guardar movimientos de tipo `DEVOLUCION_TRANSFERENCIA`.

Se corrigió la restricción de la tabla de movimientos y se permitió que el sistema terminara la compensación.

Después, la transferencia 10 cambió a estado `COMPENSADA`.

Notificación recibió el evento de devolución y generó un aviso por $150 para Ana.

Su saldo quedó en $864.

```
{
    "value":  [
                  {
                      "id":  12,
                      "clabeCuenta":  "00218000000000001",
                      "mensaje":  "DEVOLUCION_TRANSFERENCIA por 150.00, saldo resultante 864.00",
                      "fecha":  "2026-10-24T15:43:04.010716Z"
                  },
                  {
                      "id":  10,
                      "clabeCuenta":  "00218000000000001",
                      "mensaje":  "TRANSFERENCIA_ENVIADA por 13, saldo resultante 864.00",
                      "fecha":  "2026-10-24T15:27:25.909638Z"
                  },
                  {
                      "id":  8,
                      "clabeCuenta":  "00218000000000001",
                      "mensaje":  "TRANSFERENCIA_ENVIADA por 12, saldo resultante 877.00",
                      "fecha":  "2026-10-24T15:27:25.895154Z"
                  },
                  {
                      "id":  6,
                      "clabeCuenta":  "00218000000000001",
                      "mensaje":  "TRANSFERENCIA_ENVIADA por 11, saldo resultante 889.00",
                      "fecha":  "2026-10-24T15:27:25.627914Z"
                  }
              ],
    "Count":  4
}
PS C:\Users\Billie Jean\mexi-banco-s09> docker compose logs --tail=100 notificacion |
>> Select-String -Pattern "TransferenciaRevertida|DEVOLUCION"

notificacion-1  | 2026-10-24T15:43:04.010Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.notificacion.NotificacionService     : [notificacion] 00218000000000001 +Â
DEVOLUCION_TRANSFERENCIA por 150.00, saldo resultante 864.00
notificacion-1  | 2026-10-24T15:43:04.018Z  INFO 1 --- [notificacion] [ntContainer#0-1]
m.m.n.m.ConsumidorDeTransferencias       : [consumidor] TransferenciaRevertida de la transferencia 10
(evento ed03ec43-1737-4735-8bee-403ba7413d26, redelivered=false): 1 aviso(s) nuevo(s)
```

Figura 10: Aviso de devolución por $150 y recepción del evento TransferenciaRevertida.

Resultado de la devolución

El sistema pudo informar que una transferencia había fallado y que el dinero fue devuelto.

4 Mensajes duplicados e idempotencia

RabbitMQ puede entregar un mismo mensaje más de una vez.

Para comprobar este problema, se realizó una transferencia por $7 y se publicó otra copia del evento con el mismo `eventoId`.

Cuando Notificación recibió ambos mensajes, se generaron dos avisos iguales para Beto.

Los dos avisos mostraban una transferencia recibida por $7 y un saldo resultante de $643.


```
{
    "value": [
                {
                    "id": 16,
                    "clabeCuenta": "0021800000000000002",
                    "mensaje": "TRANSFERENCIA_RECIBIDA por 7, saldo resultante 643.00",
                    "fecha": "2026-10-24T15:54:34.4415792"
                },
                {
                    "id": 14,
                    "clabeCuenta": "0021800000000000002",
                    "mensaje": "TRANSFERENCIA_RECIBIDA por 7, saldo resultante 643.00",
                    "fecha": "2026-10-24T15:54:34.219993Z"
                },
                {
                    "id": 11,
                    "clabeCuenta": "0021800000000000002",
                    "mensaje": "TRANSFERENCIA_RECIBIDA por 13, saldo resultante 636.00",
                    "fecha": "2026-10-24T15:27:25.909915Z"
                },
                {
                    "id": 9,
                    "clabeCuenta": "0021800000000000002",
                    "mensaje": "TRANSFERENCIA_RECIBIDA por 12, saldo resultante 623.00",
                    "fecha": "2026-10-24T15:27:25.895526Z"
                },
                {
                    "id": 7,
                    "clabeCuenta": "0021800000000000002",
                    "mensaje": "TRANSFERENCIA_RECIBIDA por 11, saldo resultante 611.00",
                    "fecha": "2026-10-24T15:27:25.628321Z"
                }
            ],
    "Count": 5
}
```


Figura 11: Dos avisos iguales generados por una sola transferencia de $7.

4.1 Corrección de los duplicados

Para evitar este problema se implementó la idempotencia.

Se agregaron las clases `EventoProcesado` y `EventoProcesadoRepository`.

Estas clases permiten guardar en PostgreSQL el identificador de cada evento procesado.

También se modificó `NotificacionService` para revisar si el evento ya había sido recibido.

```
int registrosNuevos =
    eventosProcesados.marcarSiEsNuevo(
        evento.eventoId(),
        evento.tipo(),
        evento.transferenciaId()
    );

if (registrosNuevos == 0) {
    return List.of();
}
```

Si el identificador ya existe, no se vuelven a crear notificaciones.

El evento y sus avisos se guardan dentro de una misma transacción de PostgreSQL.

4.2 Prueba después de la corrección

Se realizó otra transferencia, ahora por $5.

Después se enviaron cinco copias del mismo evento a RabbitMQ.

El consumidor procesó la primera y reconoció las otras cuatro como mensajes duplicados.

En los registros apareció:

```
Evento duplicado; no se generan avisos
```

Al revisar la bandeja de Beto, solo había una notificación nueva por $5, con saldo resultante de $648.



Figura 12: Una sola notificación por $5 después de recibir cinco copias del mismo evento.

Resultado de la idempotencia

Aunque RabbitMQ entregó cinco copias del evento, solamente se generó un aviso por cuenta.

5 Mensajes incorrectos y Dead Letter Queue

La última parte consistió en manejar los mensajes que contienen información incorrecta y no se pueden procesar.

Para esto se configuró un exchange de errores llamado `transferencias.dlx`.

También se creó la cola `notificacion.transferencias.dlq`.

Su función es guardar los mensajes rechazados sin afectar el procesamiento de las transferencias normales.

Se modificó el consumidor para validar el JSON recibido.

Si el mensaje está incompleto o contiene información incorrecta, se rechaza sin regresarlo a la cola principal.

```
canal.basicReject(tag, false);
```

5.1 Prueba con un mensaje incorrecto

Para probar la cola de errores, se publicó un JSON incompleto en RabbitMQ:

```
{"eventoId":
```

El consumidor no pudo leerlo porque le faltaba información y lo envió a la DLQ.

Primero se comprobó que RabbitMQ tenía configurados el exchange y la cola de errores.

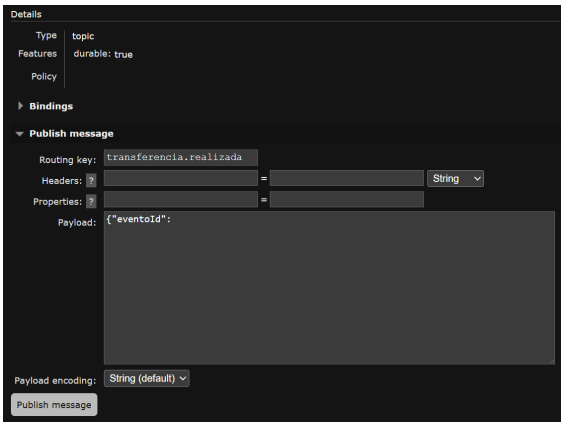


Figura 13: Cola de errores configurada en RabbitMQ.

Después se revisaron los mensajes y los registros de Notificación.

La cola principal tenía cero mensajes pendientes y la DLQ contenía un mensaje.

El consumidor también mostró el siguiente registro:

```
Mensaje invalido; se envia a DLQ
```

```
PS C:\Users\Billie Jean\mexi-banco-s09> docker compose exec -T rabbitmq rabbitmqctl list_queues name messages_read
y messages_unacknowledged consumers
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged consumers
notificacion.transferencias.dlq 1       0       0
notificacion.transferencias     0       0       1
PS C:\Users\Billie Jean\mexi-banco-s09> docker compose logs --tail=60 notificacion |
>> Select-String -Pattern "invalido|DLQ|ERROR"

notificacion-1  | 2026-10-24T23:01:41.386Z ERROR 1 --- [notificacion] [ntContainer#0-1]
m.m.n.m.ConsumidorDeTransferencias       : [consumidor] Mensaje invalido; se envia a DLQ: Unexpected
end-of-input within/between Object entries
```

Figura 14: Mensaje incorrecto almacenado en la DLQ y registro del error.

5.2 Prueba final de funcionamiento

Para comprobar que el sistema seguía funcionando después del mensaje incorrecto, se realizó una transferencia normal por $2.

La operación generó su aviso correspondiente.

En la bandeja de Beto apareció una transferencia recibida por $2, con saldo resultante de $650.

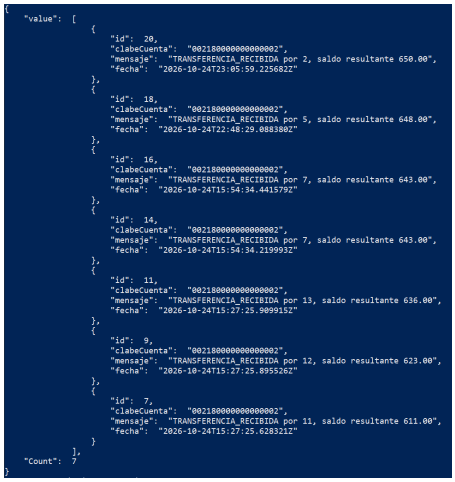


Figura 15: Aviso de $2 recibido correctamente después de probar la DLQ.

Resultado de la prueba final

El mensaje incorrecto quedó guardado en la DLQ y las nuevas transferencias siguieron generando avisos correctamente.

6 Preguntas de cierre

6.1 ¿Qué pasa si RabbitMQ está apagado?

Si RabbitMQ está apagado cuando la transferencia termina, el servicio puede guardar el estado `COMPLETADA`, pero no podrá publicar el aviso.

El *publisher confirm* permite detectar si RabbitMQ aceptó el mensaje, pero no garantiza que guardar la transferencia y publicar el evento ocurran juntos.

Para resolver completamente este problema se podría utilizar *Transactional Outbox*, aunque esa implementación era opcional en esta práctica.

6.2 ¿Por qué el ACK se envía después de guardar?

Porque primero necesitamos comprobar que la notificación se guardó correctamente.

Si enviamos el ACK antes de guardar y el servicio falla, RabbitMQ puede dar por terminado el mensaje aunque el aviso no se haya registrado.

Al enviar el ACK después, el mensaje puede volver a entregarse si ocurre un error.

En ese caso, la idempotencia evita que se creen avisos repetidos.

6.3 ¿Qué se ganó y qué se sacrificó?

Elección: Utilizar RabbitMQ para enviar las notificaciones mediante eventos.

¿Por qué? Porque una falla en Notificación ya no debe impedir que una transferencia termine correctamente.

¿Qué se sacrificó? Los avisos ya no siempre llegan de inmediato, porque pueden quedarse esperando en RabbitMQ hasta que Notificación esté disponible.